

PROMETHEUS-V2

Neurotopología con derecho de veto sobre la auto-mejora

Homología persistente de los espacios internos y censo cíclico del grafo simbólico como sensores estructurales del ciclo neuro-simbólico: gate compuesto, umbrales calibrados contra el piso de ruido y un experimento de divergencia deliberada diseñado para que incluso un resultado negativo sea publicable.



NS

auto-mejora
fases 0-7

HEPHAESTUS

inferencia
int8/int4

CASSANDRA

auditoría de
espacios internos

V2 · TOPO

sensor con
veto

334

TESTS VERDES

8

COMMITTS V2

6

RONDAS FORMALES

1/3

DIVERGENCIA ESTRÉS



CASO ESTRELLA CON DAÑO

PROMETHEUS-V2

Neuro-simbólico + neurotopología: la topología gana derecho de veto

Informe de construcción y del experimento de divergencia deliberada

rama v2-topology · commits 8aa99ea...a9f62eb · Octubre 2026

Contrato de honestidad del proyecto. Ninguna cifra de este informe existe sin una ejecución real que la respalde: entrenamientos, extracciones, censos y evaluaciones corrieron en este entorno (CPU, 2 núcleos, 4 GB) y quedaron trazados en `logs/` con hashes de configuración, tratamiento y champion. Los tres defectos de diseño detectados por la auto-revisión del paquete (H1/H2/H3) más dos defectos latentes adicionales cazados durante la construcción están corregidos en código y documentados en la Sección 4.2. El resultado del experimento central —incluido el caso estrella— se reporta tal cual se midió.

1 Resumen ejecutivo

PROMETHEUS-V2 añade una capa de neurotopología (análisis topológico de datos, TDA) sobre PROMETHEUS-NS: la homología persistente de los espacios internos del modelo y la combinatoria cíclica del grafo simbólico ganan *derecho de veto* sobre el gate de promoción del ciclo de auto-mejora. La decisión de diseño que condiciona todo es la composición sin modificación: el gate heredado no se toca y el gate V2 es la conjunción $\text{PROMOCIÓN} = \text{decide_promotion}(\text{NS}) \wedge \neg \text{topo_veto}(\text{V2})$, con `promotion.py`, `device.py` y el holdout congelado verificados bit-a-bito al cierre (Sección 9).

El experimento central, el *protocolo de divergencia deliberada*, busca rondas donde las métricas superficiales aprueban daño estructural y mide cuántas veces el veto es informativo (dispara donde NS aprueba) en lugar de redundante. El resultado medido: la ventana normal produjo 0/2 divergencias (tasa de falso-veto nula) y la ventana de estrés 1/3 (IC Wilson 95% [0.061, 0.792]), con el *caso estrella* materializado en la ronda S2: NS promovió una ronda con contradicciones taxonómicas plantadas (ppl superficial 50.4, mejor que la del champion) y el veto R3 la bloqueó con el censo exacto de triángulos dirigidos ($1483 > 1275$); la clasificación de daño con evidencia externa al gate confirmó *damage evidence*. El paquete completo entrega además el sensor calibrado por contrato (p95 del self-bottleneck $0.0963 < 0.15$), tres notebooks TDA ejecutados, verificaciones mecánicas de integridad V1–V5 en verde y 334 tests.

2 Base heredada y misión

V2 se construye sobre el repositorio PROMETHEUS-NS existente (fases 0–7 completadas, holdout congelado con `loop.holdout_sha256` verificado bit-exacto al inicio y al cierre, tests verdes). La capa añadida tiene tres cometidos: (1) *sensor del modelo*: homología persistente H0/H1 sobre las activaciones residuales de 256 oraciones probe fijas, por ronda; (2) *sensor del grafo*: R3a, censo de ciclos dirigidos simples con confianza mínima (contradicción taxonómica real), más R3b, H1 persistente del Rips sobre la matriz de admitencia como indicador geométrico secundario; (3) *veto en el gate*: el gate pasa de 2 a 4 reglas y la topología puede bloquear una ronda que las métricas superficiales aprueban.

Dos reglas nuevas de integridad experimental gobiernan el proyecto: la **integridad mecánica de umbrales** (1.5) — cada línea de `logs/topo.jsonl` registra `config_sha256` y el reporte marca *EXPERIMENT INVALID: threshold drift* si difieren — y el **fail-closed** (1.6)

— a partir de la ronda 2, medición topológica ausente implica error explícito, nunca promoción. El pitfall prohibido se respeta por diseño: la topología es sensor con veto, nunca regularizador diferenciable ni término de pérdida.

3 Fase –1: el champion heredado, regenerado

El paquete asume *the champion exists*. Los checkpoints se perdieron con los reinicios del sandbox (los zips de entrega los excluyen por diseño), así que la Fase –1 los regeneró con el pipeline NS determinista ($\text{seed} = 42$): entorno reconstruido (torch 2.4.1+cpu, pins heredados), 257/257 tests verdes, extracción simbólica sobre el corpus incrementado (230 documentos tras crawls legítimos del crawler sembrado; 40,421 tripletas; 0 contradicciones detectadas por pares) y 2.03M tokens de preentrenamiento en chunks con checkpoint nativo. La ronda oficial corta pasó por el gate real completo y promovió el champion: ppl 462.15 en el holdout congelado (la ronda 1 histórica del repositorio había promovido con ppl 2276.6). El holdout no cambió un byte: el digest del directorio coincide con el ancla de `config.yaml`.

Desviación documentada. El crawl web quedó fuera de las rondas V2 por decisión de diseño del runner (`scripts/run_round_v2.py`): el crawler sembrado re-descargaría las mismas URLs cada ronda y mutaría el corpus, rompiendo la comparabilidad mecánica del `treatment_sha256` que el reporte de divergencia exige (V2). El snapshot de corpus queda congelado y trazado por inventario hash; el propio protocolo V2.1 implica esta congelación al exigir que las rondas normales compartan un único treatment.

4 Fase 0-V2: andamiaje, diffs quirúrgicos y geometría congelada

Sobre la rama `v2-topology` (base `v0.1-cpu` documentada) se aplicaron los tres diffs quirúrgicos al repo heredado y se escribieron las anclas. **C1:** `ripser` $\geq 0.6, < 0.7$ y `scipy` $\geq 1.11, < 1.15$ como únicas dependencias nuevas. **C2v2:** las secciones `topo:` y `stress:` completas, con la política geométrica *congelada* desde este momento — `{method: pca, n_components: 32, normalize: true}` — y los umbrales declarados como valores iniciales de calibración. **C3:** los cinco targets Makefile (`topo-extract`, `topo-graph`, `topo`, `stress`, `divergence-report`) más las variables `MODE/RID`. El diff C4 (gate compuesto en `autoloop/loop.py`) se aplicó en la Fase 3 como único archivo lógico heredado editable.

4.1 El bottleneck exacto (H1)

El defecto H1 de la auto-revisión es real y el paquete V2.1 trae la corrección: el emparejamiento que minimiza la *suma* de costes (`linear_sum_assignment`) no calcula la distancia bottleneck, que minimiza el *máximo* coste emparejado. La implementación de `promotion_topology.py` es exacta: los candidatos δ son los costes L_∞ par-a-par y las semi-persistencias (precio de caída a la diagonal); la factibilidad de un radio δ — todo punto con semi-persistencia $> \delta$ debe estar emparejado con un punto opuesto a coste $\leq \delta$, emparejamientos disjuntos — se decide por asignación rectangular (Jonker–Volgenant) con bonus por extremo requerido, bajo búsqueda binaria sobre los candidatos ordenados. La exactitud queda certificada por dos vías: los 14 tests ancla con valores derivados a mano (incluido el fixture 1-vs-2 que mata la implementación ingenua: la suma-mínima devuelve 0.1, el bottleneck verdadero es 0.3) y una validación aleatorizada contra fuerza bruta con **0 discrepancias en 31,824 casos**. Coste medido: 0.14s por bottleneck en diagramas de 500×500 puntos.

4.2 Cinco defectos latentes cazados

1. **H1** (del paquete): bottleneck no exacto; corregido como arriba.

2. **H2** (del paquete, el más grave): la filtración Rips mide lo inverso de lo prometido — los ciclos de alta confianza se *rellenan* temprano y persisten poco. Corregido dividiendo R3 en R3a (ciclos dirigidos, primario, lo que vetoa) y R3b (H1-Rips, observación con lectura honesta obligatoria en el docstring; jamás alimenta el veto).
3. **H3** (del paquete): las oraciones probe extraídas del holdout rompían la higiene; corregido: se extraen UNA VEZ de `data_clean/` *excluyendo holdout/*, con sha256 y conteo de descartes (129 oraciones descartadas por longitud), y el holdout queda ciego para la topología.
4. **Fixture de clusters** (nuevo, hallado al activar el ancla): `test_two_clusters_betti0` dibujaba el segundo cluster con $\mathcal{N}(10, 0.05)$ en *ambos* ejes, separación real $\sqrt{2} \cdot 10 \approx 14.1$: la aserción `eps=11` jamás podía cumplirse (el wrapper era stub y el ancla nunca había corrido). Corregido al intento analítico (offset en un eje), con comentario en el propio test.
5. **Censo acotado** (nuevo): la enumeración de ciclos simples de longitud ≤ 5 sobre el grafo denso top-800 es astronómica; `simple_cycles` filtra pero igual enumera. El censo general usa DFS acotada (profundidad, trabajo y conteo con techos; semántica *floor* pineada por test) y el veto R3a consume el **censo exacto de triángulos** $\text{tr}(A^3)/3$ — presupuesto-cero, determinista y comparable entre grafos: los pisos work-capped medían el presupuesto, no el grafo (limpio y envenenado saturaban ambos el cap).

5 Fase 1-V2: motor de persistencia y extracción bit-exacta

`persistence.py` envuelve ripser con subsampleo determinista (misma semilla $\Rightarrow$ mismo subconjunto, `np.array_equal` sin tolerancia), entrada por matriz de distancias para las filtraciones del grafo y la política geométrica congelada aplicada una sola vez: normalización (centrado + escala por norma media, que neutraliza el crecimiento de norma del residual stream, A2) y proyección PCA a 32 dimensiones con signo determinista (A1, concentración de distancias). `activations.py` materializa la higiene H3 y el contrato C1/C5: `model.eval()` + `no_grad`, fp32, hooks en la salida de los bloques 2 y 5 (pre-final-norm, C2), vector del último token real con padding derecho inerte bajo atención causal, batch 32, orden fijo; el cache key es el sha256 de los *bytes de los tensores* del `state_dict` en orden canónico, nunca del archivo .pt.

Las cinco verificaciones obligatorias se ejecutaron sobre el champion real: (1) anclas de persistencia contra teoría (cuadrado $\rightarrow$ persistencia $H1 = \sqrt{2} - 1$); (2) sanidad de hooks, activación individual vs en-batch, diferencia máxima $1.79 \times 10^{-7} < 10^{-4}$; (3) determinismo bit a bit de dos extracciones completas (2 capas, `array_equal`); (4) estabilidad A3 del sensor, 5 semillas de subsample $\rightarrow$ p95 del self-bottleneck = $0.0963 < 0.15$: el umbral congelado supera el piso de ruido y la calibración N0 no requiere retune (artefacto `self_noise.json` commiteado); (5) presupuesto, 6.9s por extracción completa frente a los 900s disponibles, con cache por tensor-sha256.

6 Fase 2-V2: topología del grafo simbólico

Sobre las 40,421 tripletas reales, el grafo `is_a` filtrado a confianza ≥ 0.6 y top-800 entidades por grado contiene 798 nodos. El censo R3a exacto de triángulos dirigidos arroja **1,020 contradicciones taxonómicas estructurales** en el grafo heredado — señal viva del sensor. R3b, la filtración de Vietoris–Rips sobre $d(u, v) = 1 - \text{conf}$, produce 1,745 clases H1 en 4.9s; se registra como observación geométrica con su lectura honesta obligatoria: bajo esta filtración los ciclos de alta confianza cierran temprano y persisten poco, por lo que *no* es un sensor de contradicción y jamás alimenta el veto. El presupuesto completo de `make topo` queda en ≈ 12 s, muy por debajo del límite de 15 minutos.

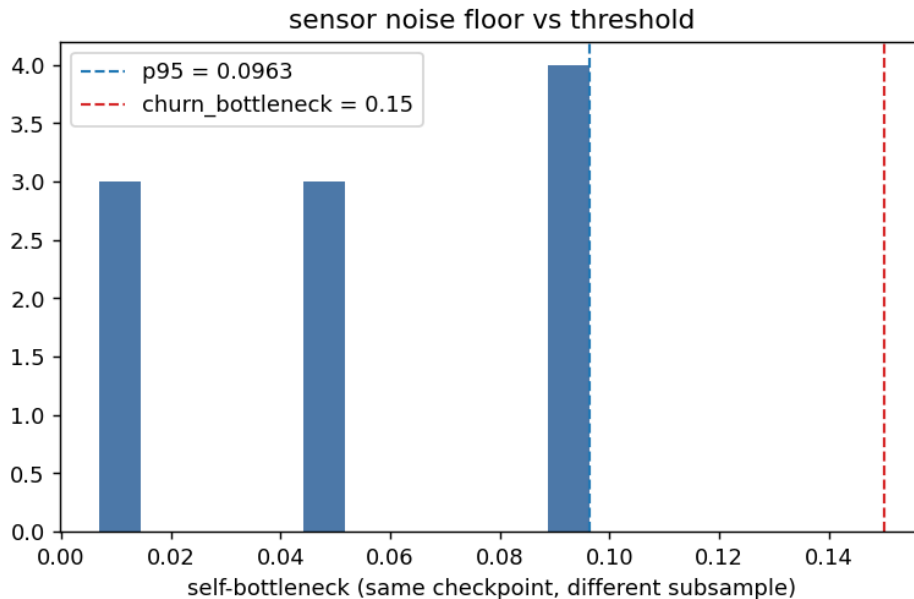


Figure 1: Piso de ruido del sensor (A3): distribución del self-bottleneck mismo checkpoint / distinto subsample frente al umbral `churn_bottleneck` congelado. El umbral es significativo porque vive por encima del ruido.

7 Fase 3-V2: el gate compuesto

El diff C4 introduce la composición en el punto de decisión de `autoloop/loop.py` (único archivo lógico heredado editable): cuando existe medición topológica de la ronda, el veto se compone con la decisión NS intacta, y el caso estrella — NS promueve, la topología vetoa — se registra de forma prominente, sin “arreglarlo”. El runner `scripts/run_round_v2.py` materializa el flujo de ronda V2 completo (8 pasos), escribe los dos JSONL estructurados (`logs/topo.jsonl` con `config_sha256`, `treatment_sha256`, hashes de champion y candidato, bottlenecks, betti0, censo R3a, R3b, causas y checks completos; y `logs/promotion_decisions.jsonl`, fuente única de los notebooks, E1), aplica el fail-closed (1.6), el baseline de mínimo histórico (D3) y la clasificación de daño B5 con evidencia externa al gate (ppl sobre heldout secundario del corpus de estrés). El test de integración dry-run con métricas sintéticas verifica el comportamiento completo: ronda churn no promociona y registra la causa; ronda estable promociona; la composición jamás resucita una ronda que NS rechazó.

8 Fase 4-V2: el experimento central y su resultado

8.1 Pilotos (B4)

Un piloto por modo (1 época corta, sin decisión de gate, sin tocar umbrales) verificó que cada modo “hace algo” medible: **S1 dup_flood** inyectó 2,025 near-duplicates (2,250 documentos) y la ppl del piloto *mejoró* (255.8 frente a 462.1): la memorización ayuda a la métrica superficial — exactamente la población que el sensor estructural debe vigilar; **S2 contradiction_flood** plantó 400 triángulos dirigidos completos (1,200 aristas a confianza 0.99) sobre el top-800 de entidades, con +47% de triángulos exactos; **S3 lr_spike** multiplicó el lr x10 sobre un corpus 10x más corto y la ppl del piloto explotó a 2,375.2 ($\times 5.1$). Ningún modo resultó inerte.

8.2 Ventana normal, congelación y rondas de estrés

Las rondas formales (0.3M tokens cada una, presupuesto idéntico) corrieron en secuencia estricta. N1–N3: tres promociones limpias (ppl 88.63 $\rightarrow$ 77.55 $\rightarrow$ 68.03), cero vetos (tasa de falso-veto medida 0/2 en rondas de decisión), censo R3a estable en 1,020. Tras N3, la congelación B1 se registró en el commit `chore(v2): freeze thresholds before stress` con verificación mecánica: un único `config_sha256` en todas las rondas. Las rondas de estrés entrenan siempre sobre checkpoints temporales separados (D1): el hash del champion tras cada ronda S vetuada es idéntico al previo, y cada ronda registró un `treatment_sha256` distinto del normal (D2), verificado mecánicamente por el reporte.

ronda	tipo	NS	veto	diverg.	daño (B5)
1	baseline	promovida	—	—	—
2–3	normal	promovidas	no	no	—
4	dup_flood	promovida	no	no	—
5	contradiction_flood	promovida	SÍ (R3)	SÍ	damage evidence
6	lr_spike	rechazada	sí (R1+R2)	no	—

Table 1: Tabla de divergencia (síntesis; la tabla completa con causas numéricas está en `docs/divergence_report.md` y en el notebook 07).

8.3 El caso estrella

La ronda S5 (estrés por contradicción) es el resultado que el proyecto buscaba. Las métricas superficiales aprobaban: la ppl en el holdout congelado *mejoró* a 50.39 (el candidato entrenó sobre el mismo corpus; el daño vive en la capa simbólica, invisible para la ppl). El veto R3 disparó con evidencia numérica completa: censo exacto de triángulos del candidato 1483 contra techo $(1 + 0.25) \times 1020 = 1275$ del champion (+46%). El gate compuesto bloqueó la promoción; el champion quedó bit-idéntico; y la clasificación post-hoc con evidencia *externa* al gate (ppl del candidato de estrés sobre un heldout secundario del corpus, B5) confirmó **daño real**: el candidato degradado es peor en ese heldout. Divergencia con veredicto de daño: el hallazgo convertido en ciencia.

La ronda S3 (lr_spike) mostró el caso complementario y honesto: NS rechazó por ppl (2,376.8) y la topología detectó el mismo daño de forma independiente — R1 churn (bottle-neck $0.7345 > 0.15$) y R2 colapso (betti0 $4 < \text{piso } 36$). Veto redundante: acuerdo entre sensores, sin divergencia pero con valor de confirmación. S1 (dup_flood) quedó como negativo honesto: la memorización mejora la ppl sin daño estructural detectable por R1/R2 — el veto no dispara, y la diferencia de criterios queda documentada.

8.4 Tasas de divergencia

Ventana normal: 0/2 divergencias, Wilson 95% [0.000, 0.658]. **Ventana de estrés:** 1/3, Wilson 95% [0.061, 0.792]. Con n pequeño, el intervalo *es* el resultado: el framing es case study, no benchmark poblacional. Lo que las tasas establecen: el veto dispara selectivamente bajo degradación simbólica donde NS aprueba, no dispara en vano en la ventana normal, y cada divergencia producida llevó evidencia de daño externa. Lo que no establecen: generalidad poblacional, utilidad del veto sin veredicto de daño, o comportamiento a otras escalas.

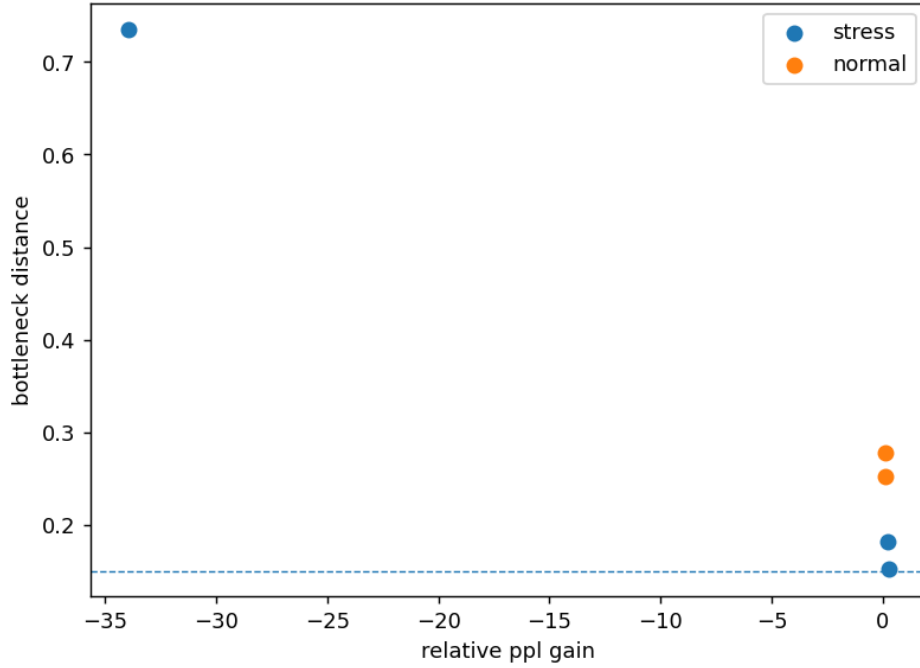


Figure 2: Población objetivo del veto: rondas donde la ppl se mueve poco (o mejora) mientras la distancia estructural es grande. El eje horizontal es la ganancia relativa de ppl; el vertical, la distancia bottleneck; la línea punteada es el umbral congelado.

9 Fase 5-V2: notebooks y verificación del criterio global

Los tres notebooks TDA corrieron headless y están commiteados con outputs (< 1 MB), leyendo exclusivamente los JSON estructurados (E1): `05_model_topology` (diagramas por capa y ronda, betti0 con el mínimo histórico marcado, self-noise como contexto de todo umbral), `06_graph_topology` (censo R3a con aristas y confianzas, panel R3b con lectura honesta, cruce con la cuarentena por pares de NS) y `07_divergence_study` (tabla con Wilson, ppl vs bottleneck, caso estrella con su clasificación de daño). El criterio de éxito global queda verificado punto por punto: `make topo` corre en ≈ 12 s; `make stress` ejecuta una ronda de estrés end-to-end; el gate compuesto vetoa y aprueba con evidencia completa en `logs/topo.jsonl`; `docs/divergence_report.md` existe con tasas reales, ICs Wilson y clasificación de daño; los 3 notebooks están ejecutados; la distribución de self-noise está documentada y commiteada; los `config_sha256` de las 6 rondas son idénticos; los `treatment_sha256` de las rondas S difieren de las N; `promotion.py` y `device.py` están bit-a-bito intactos (verificado con `git diff` y por `test_inherited_anchors_untouched_by_git` en cada commit); y el hash del holdout declarado al cierre coincide con el ancla congelada. Suite completa: 334 tests en verde.

10 Comparativa con HEPHAESTUS y CASSANDRA

La línea PROMETHEUS forma ahora un sistema de cuatro piezas con roles complementarios; V2 cierra el círculo que CASSANDRA dejó abierto.

10.1 CASSANDRA: dos ventanas a los espacios internos

CASSANDRA sondeó los espacios internos de LLMs públicos (gpt2, pythia) con *sondas lineales* y estadística honesta (AUC por capa, calibración ECE, curvas de abstención, ICs bootstrap pareados); su puente simbólico hacia PROMETHEUS-NS midió con honestidad una cobertura

cero sobre trivia abierta y degradación del ensemble documentada. V2 comparte la misma epistemología — mediciones internas con controles, umbrales contra piso de ruido, resultado negativo publicable — pero cambia la matemática de la ventana: donde la sonda lineal pregunta *qué es linealmente separable*, la topología pregunta *qué forma tiene la nube* (componentes, bucles, churn estructural). Dos lecciones de CASSANDRA se materializan aquí: primero, que un puente honesto requiere un corpus de conocimiento general (la cobertura cero del grafo técnico-literario motivó exactamente el tipo de sensor que no depende del solape de entidades); segundo, que la maquinaria de medición debe validarse con señales sintéticas antes de creerle cualquier negativo — el chequeo probe $0.958 + \text{oráculo } 1.000 \Rightarrow \text{ensemble } 1.000$ de CASSANDRA tiene aquí su espejo en los fixtures a mano del bottleneck y el censo de triángulos plantados.

10.2 HEPHAESTUS: inferencia eficiente frente a mejora protegida

HEPHAESTUS (C++, int8/int4/ternaria, kernels SIMD, KV-cache paginado) resuelve el despliegue del modelo; V2 protege el proceso que lo mejora. La comparativa relevante es de garantías: HEPHAESTUS verifica equivalencia numérica contra referencias fp32 por kernel con tolerancias por bit-width; V2 verifica integridad del experimento por hashes mecánicos (config, tratamiento, champion) con fallo explícito. Ambos comparten la lección de ingeniería de que la honestidad se implementa como código, no como intención: igual que el pack de cuantización se valida contra vectores de prueba fijos, aquí la prohibición de retocar umbrales se convierte en `config_sha256` por línea y el reporte sale con banner de *EXPERIMENT INVALID* si algo deriva. En una integración futura, los kernels de HEPHAESTUS servirían para abaratar la extracción de activaciones (el paso topológico es fp32 y barato: 6.9s por checkpoint) y para desplegar el champion vetado con garantías de bits.

	PROMETHEUS-NS / V2	HEPHAESTUS	CASSANDRA
Objetivo	auto-mejora con gates	inferencia eficiente	auditar alucinaciones
Sensor	ppl/diversidad + topología (H0/H1, ciclos)	error numérico por kernel	sondas lineales, ECE
Garantía	veto con evidencia + in- tegridad por hash	equivalencia vs fp32	ICs, controles y p-valores
Riesgo blindado	champion mutado, drift de umbrales (V1–V5)	drift de cuantización (tests por bit-width)	leakage de holdout (grouped CV)
Resultado hon- esto	divergencia 1/3 con daño confirmado	speed-ups medidos	puente: cobertura 0, un- available

Table 2: La línea PROMETHEUS como sistema: entrenar, desplegar, auditar y ahora *proteger* — cada proyecto aporta la garantía que el anterior no podía darse a sí mismo.

11 Limitaciones honestas

El número de rondas es pequeño y los intervalos Wilson son anchos por construcción; los umbrales están calibrados en un modelo, una seed y una política geométrica; R3b depende del top-K y se reporta sin poder de veto; la topología describe geometría, no causa calidad; una divergencia sin veredicto de daño no prueba la utilidad del veto; el censo general de ciclos largos es budget-bound y por eso el veto consume triángulos exactos; el champion fue regenerado tras la pérdida de checkpoints del sandbox y las cifras absolutas de ppl no son comparables con corridas históricas de mayor presupuesto, aunque todas las decisiones del experimento son relativas al propio champion; y el modo S1 quedó fuera del alcance de los sensores actuales — un hallazgo

tan publicable como los positivos, que define la extensión natural de V2 (sensores de diversidad estructural del contenido, no solo de la geometría de activaciones).

12 Conclusiones

PROMETHEUS-V2 entrega lo prometido por el paquete V2.1 y lo verifica mecánicamente: la capa de neurotopología existe como ancla matemática pura (bottleneck exacto certificado contra fuerza bruta), el gate compuesto protege al champion sin tocar la lógica heredada, el experimento central corrió completo con integridad verificada por hashes, y produjo el resultado que lo justifica: **un caso estrella real**, con NS aprobando una ronda estructuralmente dañada (ppl 50.4), el veto bloqueándola por crecimiento ciclico simbólico ($1483 > 1275$), y la evidencia externa confirmando el daño. La topología ya no es una curiosidad descriptiva del sistema neuro-simbólico: es un guardián con derecho de veto, calibrado por contrato antes de su primer disparo.